Lab04

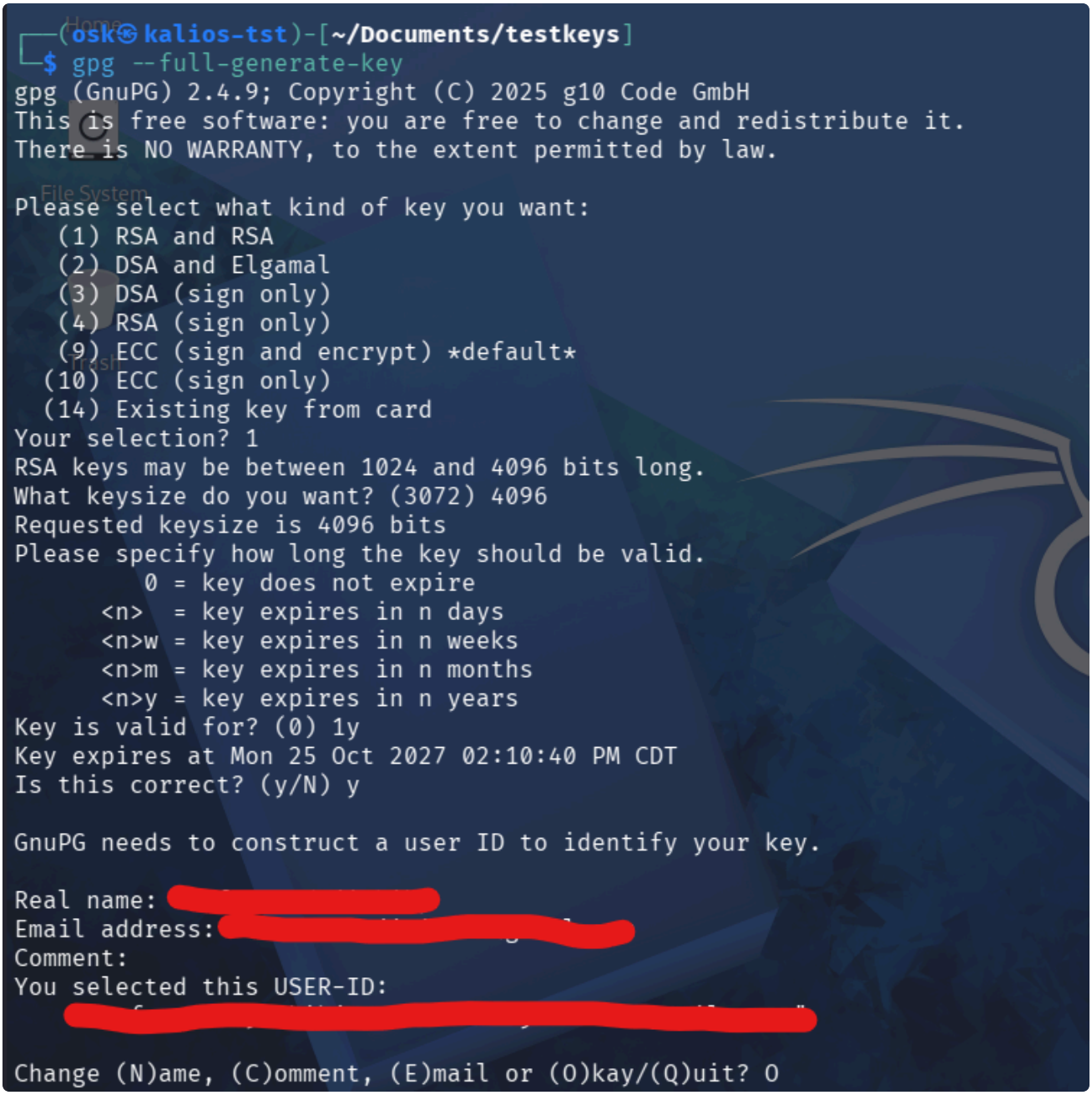
#cyber-security

1 - Generating GPG Key Pair:

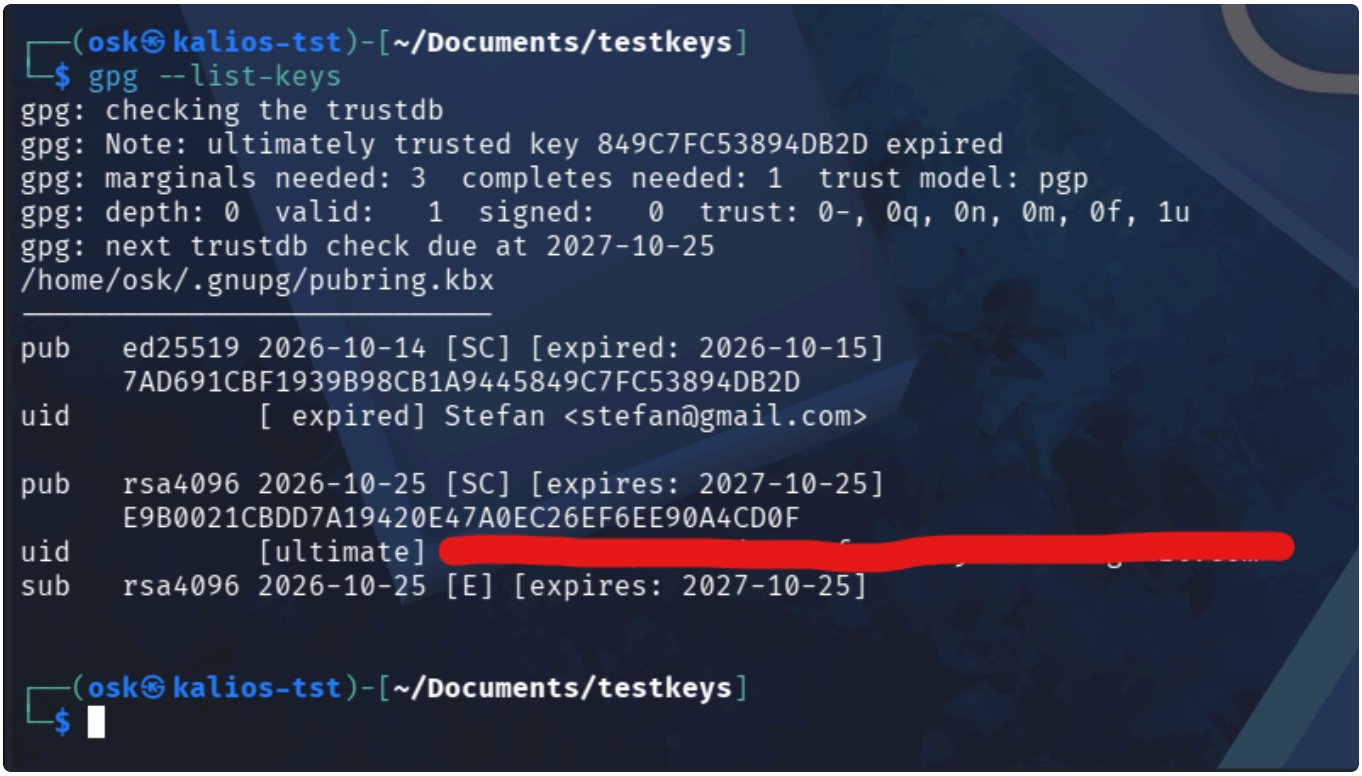
We first generate the GPG key pair using the command `gpg --full-generate-key`, which will proceed to ask us various question regarding the generation of the keys.
We select the following settings:

1. RSA and RSA
2. Length of 4096 bits
3. Key is valid for 1 year

4. Provide the name and email address information



We can then verify the keys were successfully generated using `gpg --list-keys`



2 - Export and Import Public Key:

Exporting our public key:

```
┌──(osk㉿kalios-tst)-[~/Documents/testkeys]
└─$ gpg --armor --export OUR_EMAIL_HERE > example_pubkey.asc

┌──(osk㉿kalios-tst)-[~/Documents/testkeys]
└─$ ls
example_pubkey.asc

┌──(osk㉿kalios-tst)-[~/Documents/testkeys]
└─$ cat example_pubkey.asc
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQINBGreVEwBEAC+hHDIKNBk0jX3GpIV7Gs/hquC7rDoLUFGzPIniHCr8G2uJCJJ
UFooEPG3Qgg8k/N+eGVr1Jk1lzwE9F3xiPvEwd38kan8Ebx4rqyLrqemV835M/uj
P2sFB5Hq9AGp5ilkKHE8CM36BmRhvr3wVi5XDsjyimgoYMd4oBOoC7Vde0YaRO0y
0OKArzxWseLnADuWj7UkmJOMkMKold2veOoUbyveEYKuWLkT/bSOghE6UyxJlVU/
scE2rNVoaU8CfW935uEz3GyeabPglHU0SBPtxvIwDfGlfbX0kSNK4L2eOkpHfQ9z
DIzukPiYDkkp5unelWCtSyiPLHJRfixGaPz2icVH3XV+iS7sWHXUN7GRZEyyKdIq
a89wn+bYGm3IkbRatlattSF3GIwFWmHDCLpseCGreHPJ34ckm2KumnLDZlVSpnJo
vsh4Xk8vciRfT5HZ5o/BaXcY8DgpZVTJX4e7QBkiGUdWJl/nrKHF6EXpopgw1u4G
M19IeriL7zdqrBCa3M1QeaDfrDLnMzLSa8sRpT+WUbonYyyip9qwb+7TYkHGQEYV
hXh1h05J/hr0OkO427YCQYvtOKpy9jtmmZ2z1ff5bbGpgtygzsu4fd5t4QScavM9
```

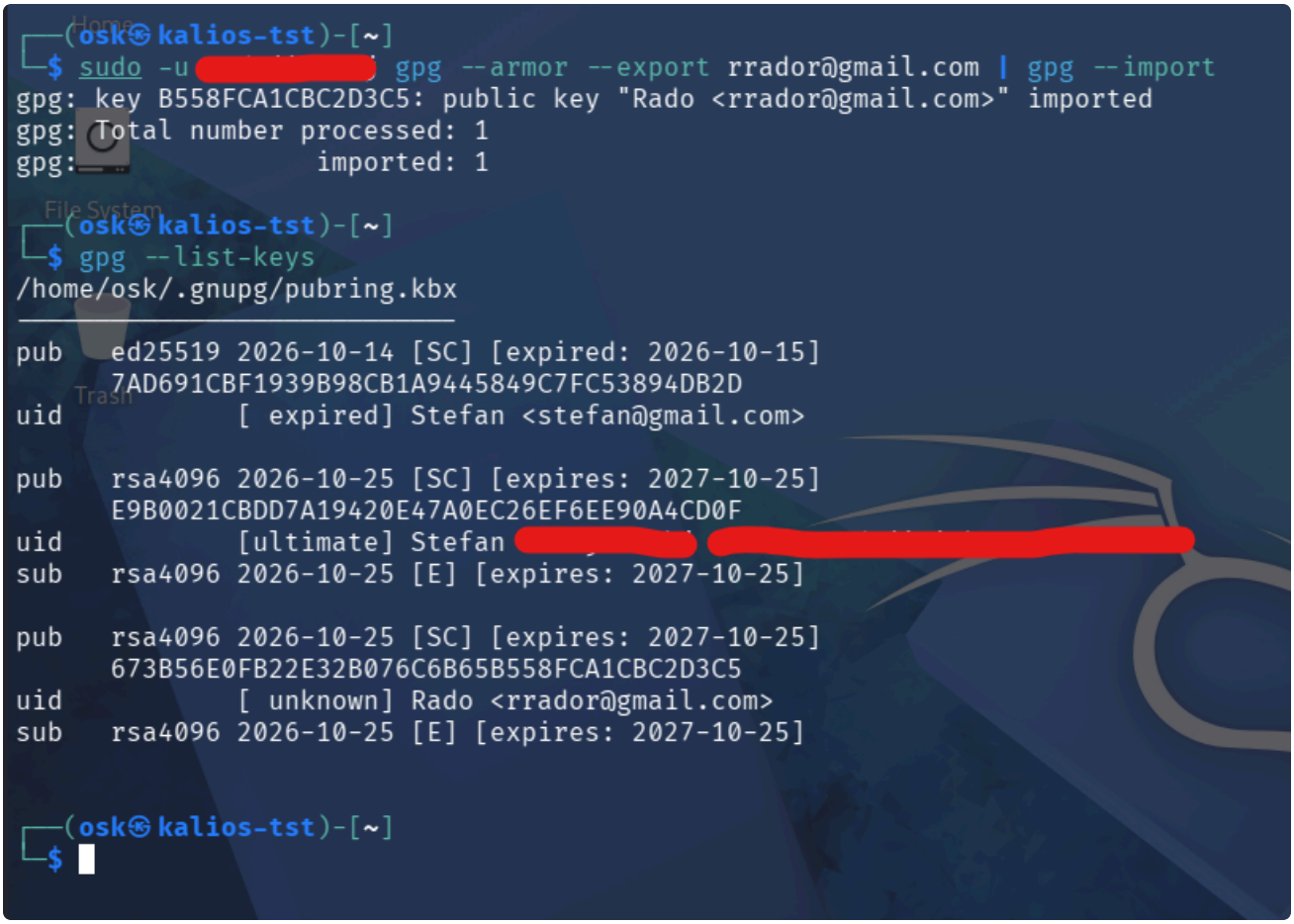
```
Ba9VGBbs8GFwUDt8wCDbMAlYszljzahbTRu2DftkJCPcglPlHUg6sWK9lwARAQAB
tDFTdGVmYW4gUmFkb2ppY2hpa2ogPHN0ZWZhbnJhZG9qaWNpY2syM0BnbWFpbC5j
b20+iQJUBBMBCgA+FiEE6bACHL3XoZQg5HoOwm727pCkzQ8FAmreVEwCGwMFCQHh
M4AFCwkIBwIGFQoJCAsCBBYCAwECHgECF4AACgkQwm727pCkzQ9sgBAAsou5rAiS
+HaDtCBKkLdv0jTT3M5CkRU1J6k2zsvmy02IIifgJZ1ub7neDWmT8vfRgP7GemIa
CYFR7kPzroGPXsXoJC6Tjzxe04wYUtMuFYjGsvMrj+gVKJ9IQz22pJk2sInUp586
DreStUHPCPyJHA/PI+HqxdVvI0UBKaiv9DdwTshUI4rmKMXaPspDv+ZyCH4lxOF+
TQPH/Y3X8cepd+hkCnkH5kmObCwv2vWaNihC4hVpcvhx8OWx9kQt1T8QJDUs71d/
hNs8Ysk/sanuEEMB6lOthcxKNoCTuXVPznp3+88qHtH9XDjrW1CH5dTBU6rmdHq3
NvintUXwWBERhAkgHwRoyJir9hbJlDC9FQyAQJzDJ6X0kJnbfFlSjA9brlr5r9Wi
hZ2UIDjOmlLg9m2YLAwDcz/xZEpZaC6WGxKFPA+Ct/PI9olzXO7XvNq1Kk83vP66
bhQch2UWOp8/urlpWlVA7K69mDew8/paNiM8I2sMQIGdJGGnr3r62mfqpMvi/Qjm
sJ9D374GT+vHMBXSuL+dl4PdB0Zv3KziUpD7CQl2KHjMHqG0hrAh6VgMGLwDx6WR
xT3wyGYZAOK4mqi7giLHnyXrAIfkBkTZv3mi7zH9HNaAp6qF/so+gvuiob9zjUN2
geJSV2qsUjnRo2Cbl6GiKxoXrBIs1v5J8DS5Ag0Eat5UTAEQANL4nEkBMcQgjayO
a4ewTnSA2EeowDOdVTeHyy6mooRWheegw8TtRitOQCHvAH83PELT5+Qs2wKo9FQ5
51hVx6//9hM3J3sjzlry+eMIgiIoeZU3iXEa+SuPD7GXDGUmyAB97dOvFU1jRfFi
bs4MwWSyRchlCuuvtK4yI2jV64YNZeovpAQbm8MB2aAWK04QFXMyx78ob9oYkJ/B
JcP/y0sf52Lsf+ERStdojlaeqM9mHSRV73J5MIufl+n/+//mEVLShPvxghrLuCLS
wF3FaLSGhFrcn8o3YimtD9by63re/THwnQfM51E5tsDOzVjm3+w2Z1emHucPP2mu
Clmj1OWGyId1FJqCH64ihEWjEJTwahnGobEazMfOR8qJ7ln6k/GSnN6BsAnyWGUD
FhQ7rGd4MhxAF6O9iMHMALEV49CFIHVz49ti1m2juTSTdgIK1R9uMaduWjKFbPl9
ByZxfQ+I3MR7xp9wEEl8MS+TyORChdg2sbR09juPEbkNKPHvMuKt+NJgewr8ISL/
zvnw5AoyxAHXuQF8siY04KT68d+mu2JsvgkGrRO93x4s8cOZSZ4zkHjfU2ZXYish
dLy9yRjaHKsCGjB6GClZQYf600x4LqUA7CjtqjkFzasAWdudI4J72wumVRYtwPks
P3z2aW6Z98KGiOEHU+/JgTvxnH79ABEBAAGJAjwEGAEKACYWIQTpsAIcvdehlCDk
eg7CbvbukKTNDwUCat5UTAIbDAUJAeEzgAAKCRDCbvbukKTND4fTD/4juVAEpa9B
yHRS4QxHF07LRLRC1O40G5fleMLiPqr2pSKVeA31wLf0Iq6T+b8VTsU3fDoq4iYZ
oaO6jBKdKeyhBKtMYy9jDb3OTGd8H7P3P/m3lIFWDiswnHhfhR2mKWibuDcJM/zk
kEtm7Xc5CDq9eJ30n5AaJBR+TcXvCiVKHrAg1Ezvyta6nyFdV36BzelgH4fYq7ul
7j6niq5Nmgyc7dyFnyYF07KkZmJCH7FS3ekrb9zgpg3whDtHOsp+bG8qloN/GGsb
PGSy+MMzGRI/BYeHldXxeHTCfn9N9l35d5vNVWvn+SEJFi2iVF+knnJMYMvJ7tTc
TVsm4pbaQCeneGnB4jMon1tzD7CKE+7onNOUKGPAErVgd79dQ/VjfwShYG0Be1jY
G+cmHj5eaVO+WkI3RWlLN3D8Zkfb2BD/phZCKx+M4E1036OG8gUdsduZ1jm+Myn2
npp3KA4PqV5qnT046avQ9zuj4OiZpMobZYhw6MCvc4Os9MUle5/Nmf5swY+jZDP2
ZLxb70Ng7jRM0J9dd4fJ7u+UDjvqk6YbWrizDn9ztJLa0JCMFPvVASQcpWQcTpO0
RdWK+4rDamfr8oSuXF6qYhMv44WiN1z+4tNUiZ4EOJSIfTB5Osrld1AASWFOD7yI
SKNOucrh8LJYUd7iRebaNXy7huAJFGb0Pw==
=g7+i
-----END PGP PUBLIC KEY BLOCK-----

┌──(osk㉿kalios-tst)-[~/Documents/testkeys]
└─$
```

Importing a Foreign Public Key:

We can login as a different user and repeat the process in the first, step, after which we log back into our main user and can then import the generated key for our main user via: `sudo -u SECOND_USER gpg --armor --export 2ND_USER_EMAIL | gpg --import` where:

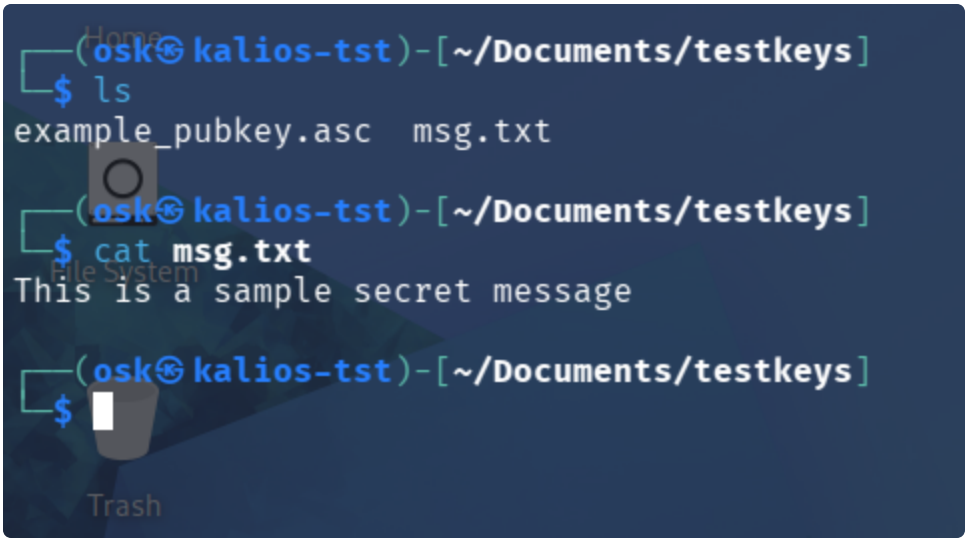
- We use `sudo -u` to provide the user to use to execute the first command, where we do the same steps as the previous section. Then we pipe that output to the `gpg --import` command.
  We can check the success of the command, verify by running `gpg --list-keys`:



Encrypting and Signing:

We create a sample text message:
`echo "This is a sample secret message" > msg.txt`

We verify this sample file has been created



We can then encrypt the message and verify that it was successfully encrypted by running the commands:

```
gpg --encrypt --armor --sign --recepient THE_SECOND_USER msg.txt
```

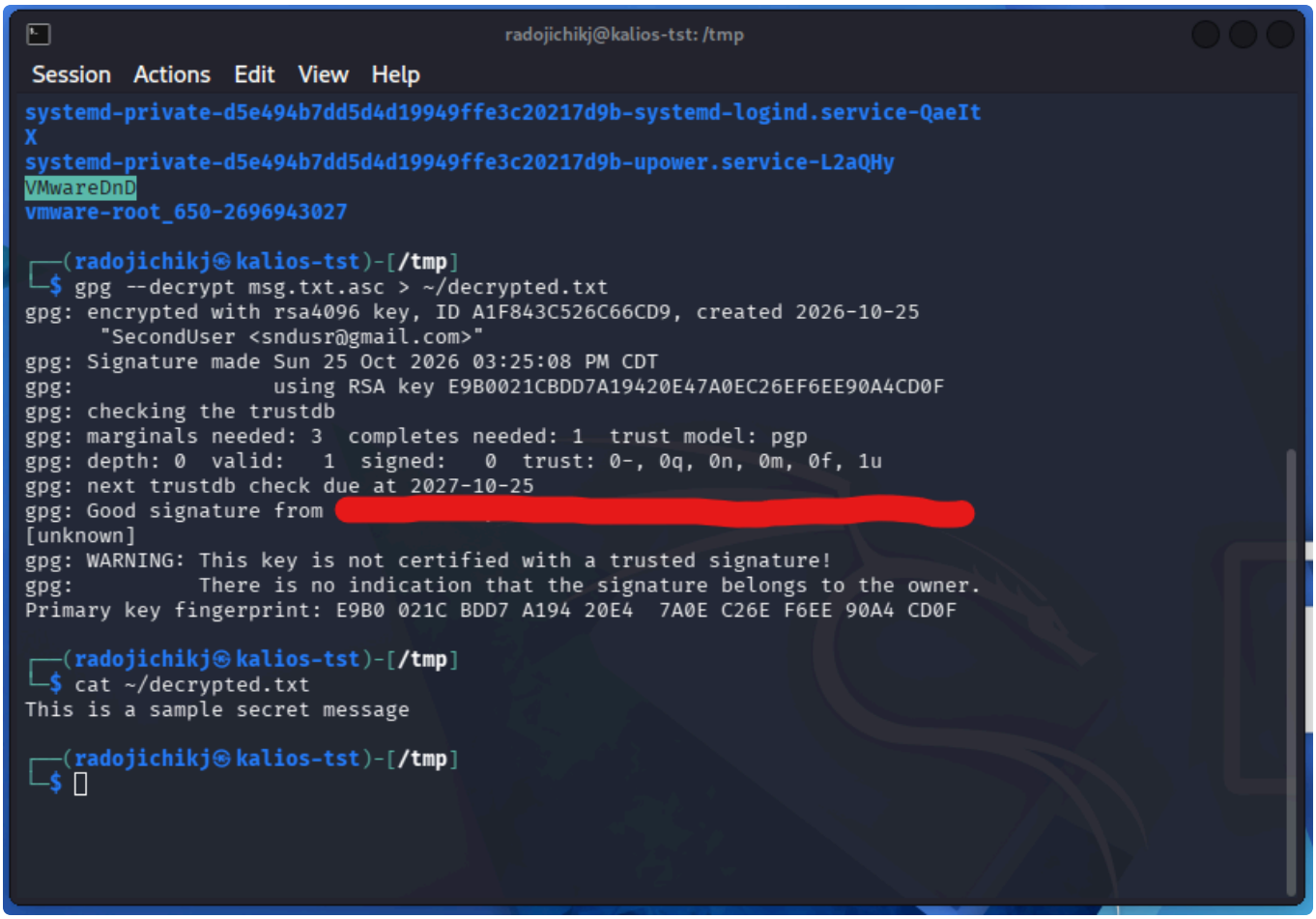
then print out its contents:



3 - Decryption and Verification

note: Both sides need to have their key pairs generated, their public keys exported then imported by the opposing party in order for this process to work properly from start to end.

We utilize the `/tmp` folder to share the secret file. We can decrypt it as the second user utilizing the command `gpg --decrypt msg.txt.asc > ~/decrypted.txt`
If we have the opposing party's public key imported, we should have the following output upon running the command.



Extra Tasks:

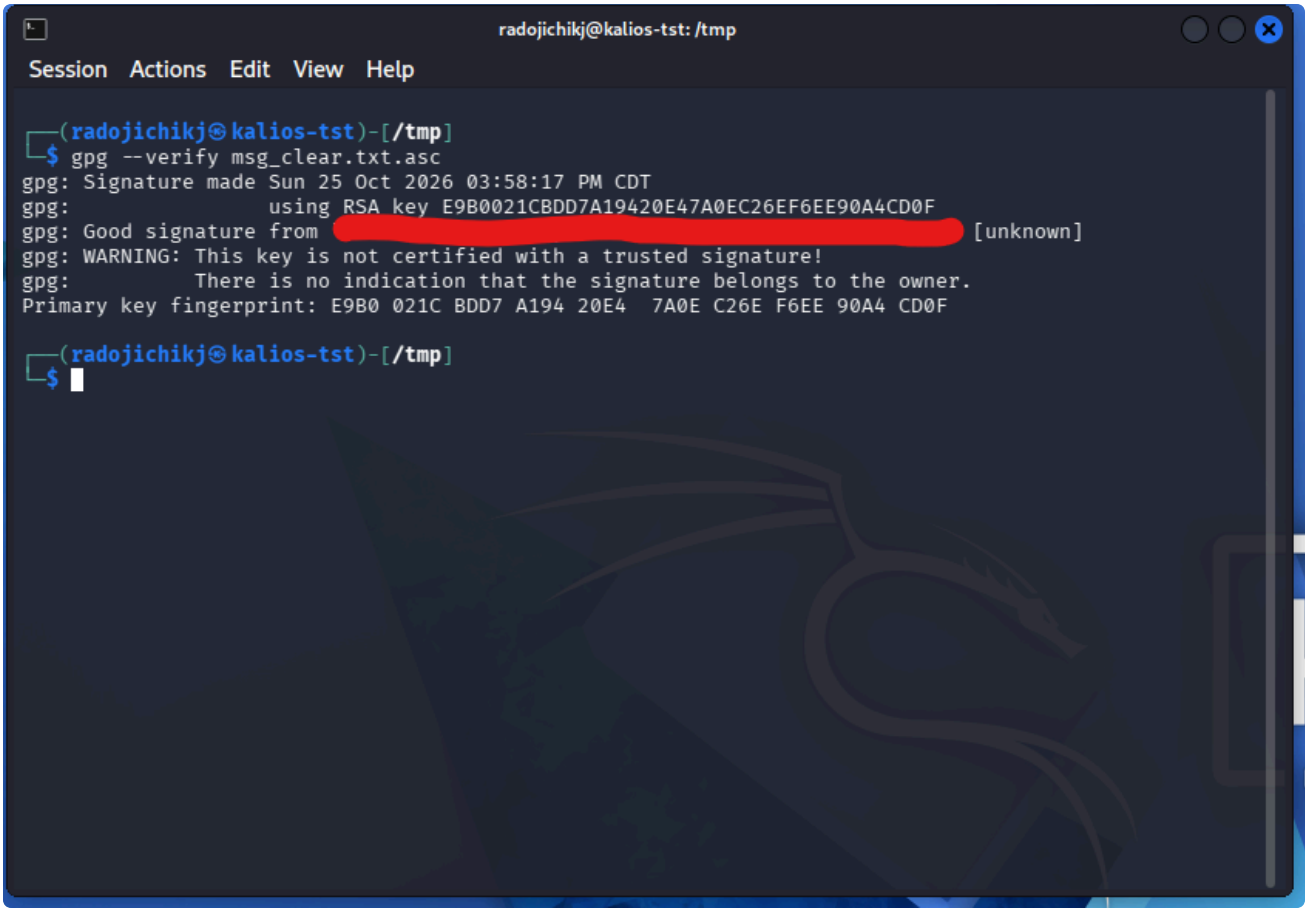
1. Create revoke certificate



2. Generating a clear-sign message



3. Verifying signatures



Closing Thoughts:

1. Difference between encryption and signing : Encryption provides confidentiality by protecting the content from being read by everyone, while signing provides integrity and authenticity (that the message has been un-altered and real - coming from the real source specified)
2. Role of public and private key - Public key encrypts the messages user A sends to user B, then users' private key decrypts the message that user A sent to them using their public key and vise versa. (Private keys within RSA are also used for signing while the public for verification)
3. What happens when an encrypted file is modified : The decryption will fail due to failed integrity checks. These checks are implemented in different ways under different methodologies but in the case of GPG it utilizes a SHA1 hash of the contents and it appends it to the contents before encryption.